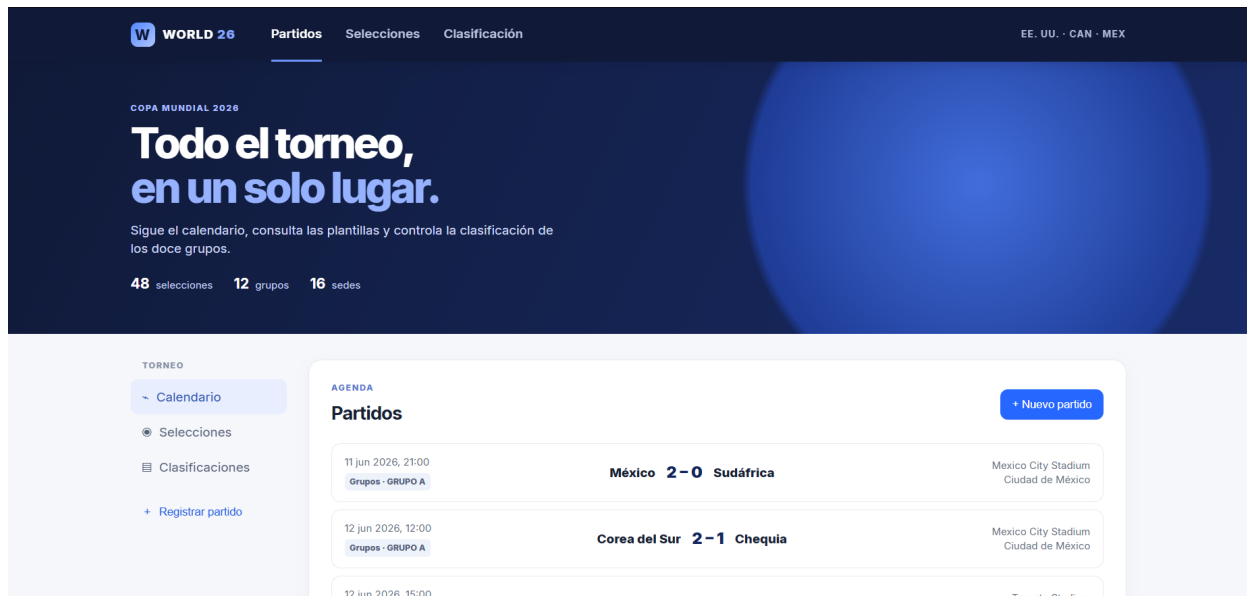


COPA MUNDIAL 2026

Proyecto Mundial 2026 API Web

Cliente web en JavaScript vanilla con API REST en PHP, base de datos MySQL/MariaDB y diseño final refinado con apoyo de IA.



Vista final del cliente web: agenda de partidos, cabecera del torneo, navegación lateral y estética visual renovada.

Autor	Alex Punzano Gil
Tipo de proyecto	Desarrollo web full-stack académico
Tecnologías principales	HTML5, CSS3, JavaScript vanilla, PHP 8, PDO, MySQL/MariaDB
Apoyo de IA	Codex e IA generativa utilizada principalmente para iterar y refinar el diseño CSS, manteniendo revisión y validación manual.
Repositorio	https://github.com/alexpunzi/mundial-2026-api-web
Entregables	Cliente web, API REST, scripts SQL, README y documentación final en PDF

Proyecto preparado como pieza de portfolio. La ejecución completa requiere servidor local con Apache con PHP y MySQL; GitHub Pages solo sirve el cliente estático y no ejecuta el backend PHP ni la base de datos.

1. Resumen del proyecto

El proyecto consiste en una aplicación web para consultar y gestionar información relacionada con el Mundial 2026. Combina una interfaz de cliente construida con HTML, CSS y JavaScript vanilla con una API REST desarrollada en PHP y una base de datos relacional en MySQL/MariaDB.

El objetivo técnico principal fue practicar el consumo de servicios web desde el cliente, la gestión de rutas REST, la validación de formularios, las operaciones CRUD y el calculo de clasificaciones a partir de resultados registrados.

Area	Descripción
Cliente web	Interfaz para seleccionar grupos, consultar selecciones, visualizar partidos, ver clasificaciones y registrar nuevos partidos.
API REST	Capa PHP que expone endpoints para selecciones, jugadores, estadios, entrenadores, partidos y clasificaciones.
Persistencia	Modelo relacional con tablas para entrenadores, estadios, selecciones, jugadores y partidos.
Formato de datos	Respuestas JSON con códigos HTTP y estructura común para resultados correctos y errores.
Diseño final	Interfaz renovada con cabecera fija, hero visual, tarjetas, tablas limpias, sidebar y paleta azul corporativa.

Objetivos tecnicos

- Construir un cliente web funcional sin frameworks, usando JavaScript vanilla y peticiones XMLHttpRequest.
- Diseñar una API REST capaz de consultar, crear, actualizar y eliminar recursos principales.
- Validar datos de entrada antes de insertar o modificar información en la base de datos.
- Calcular clasificaciones por grupo a partir de partidos con resultado registrado.
- Preparar el proyecto para GitHub evitando credenciales locales y archivos innecesarios.
- Mejorar la presentación visual del producto final mediante iteraciones controladas de CSS con apoyo de IA.

Arquitectura de la aplicación

Cliente web HTML, CSS JavaScript vanilla XMLHttpRequest	HTTP/AJAX Peticiones y respuestas JSON	API REST PHP 8 Rutas limpias Validación	SQL/PDO Consultas preparadas	Base de datos MySQL/MariaDB Selecciones, jugadores, partidos y estadios
--	--	--	------------------------------------	--

2. Funcionalidades principales

La aplicación permite consultar datos del torneo y realizar operaciones de escritura controladas desde el cliente. La separación entre frontend y backend facilita que la interfaz consuma información mediante peticiones HTTP y reciba respuestas JSON.

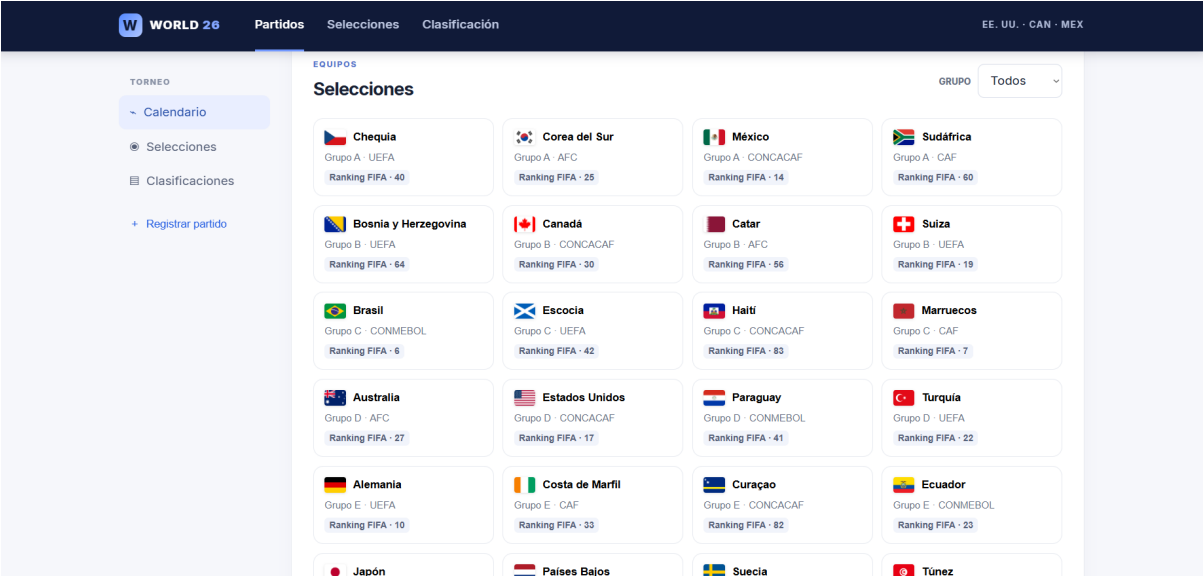
Funcionalidad	Descripción
Consulta de selecciones	Listado completo o filtrado por grupo, con acceso a la ficha de cada selección.
Detalle de selección	Consulta de grupo, confederación, ranking FIFA, entrenador y jugadores asociados.
Gestión de partidos	Listado de partidos, registro de nuevos encuentros y asignación de estadio, fecha, fase y selecciones.
Registro de resultados	Actualización del marcador de un partido ya creado.
Clasificación por grupo	Cálculo de puntos, victorias, empates, derrotas, goles a favor, goles en contra y diferencia de goles.
Interfaz de torneo	Navegación entre agenda, selecciones y clasificaciones, con filtros por grupo y componentes visuales consistentes.

Endpoints representativos

Metodo	Ruta	Uso
GET	/api/selecciones	Listar selecciones. Acepta filtro por grupo.
GET	/api/selecciones/{id}	Consultar detalle de una selección.
GET	/api/selecciones/{id}/jugadores	Listar jugadores de una selección.
POST	/api/selecciones	Crear una nueva selección.
PUT	/api/selecciones/{id}	Actualizar una selección existente.
DELETE	/api/selecciones/{id}	Eliminar una selección si no tiene partidos asociados.
GET	/api/partidos	Listar partidos; admite filtros por grupo, selección o fase.
POST	/api/partidos	Registrar un nuevo partido.
POST	/api/partidos/{id}/resultado	Registrar o modificar el resultado de un partido.
GET	/api/grupos/{grupo}/clasificación	Obtener clasificación calculada del grupo indicado.

3. Interfaz final y diseño visual

La versión final sustituye una composición más basica por una interfaz con identidad propia: cabecera fija, marca World 26, navegación principal, panel lateral, tarjetas con bordes suaves, jerarquía tipografica y una paleta azul coherente con el contexto del torneo.



Vista de selecciones: tarjetas por equipo, filtro de grupo, banderas, confederacion y ranking FIFA.

Elemento	Implementación final
Cabecera y navegación	Barra superior oscura con marca, menu de secciones y países anfitriones como referencia visual.
Sidebar	Navegación secundaria para calendario, selecciones, clasificaciones y registro de partidos.
Tarjetas	Componentes reutilizables para selecciones, con bandera, nombre, grupo, confederacion y ranking.
Tablas	Clasificaciones con columnas compactas, pesos tipograficos claros y separación por grupos.
Paleta y espaciado	Uso dominante de azules, fondos claros, radios amplios, sombras suaves y espaciado consistente.

La IA se empleó como apoyo de iteracion visual, no como sustituto de la implementación: las propuestas de CSS se revisaron, adaptaron y probaron manualmente sobre la estructura real del proyecto.

4. Clasificaciones y experiencia de usuario

La pantalla de clasificaciones organiza los grupos en tarjetas/tablas independientes. Este enfoque mejora la lectura porque cada grupo mantiene su propia jerarquía: nombre del grupo, equipos ordenados, partidos jugados, diferencia de goles y puntos.

W

WORLD 26

Partidos
Selecciones
Clasificación

EE. UU. · CAN · MEX

TORNEO

~ Calendario

● Selecciones

☰ Clasificaciones

+ Registrar partido

FASE DE GRUPOS

Clasificaciones

Grupo A

Equipo	PJ	DG	PTS
1 México	3	+6	9
2 Sudáfrica	3	-1	4
3 Corea del Sur	3	-1	3
4 Chequia	3	-4	1

Grupo B

Equipo	PJ	DG	PTS
1 Suiza	3	+4	7
2 Canadá	3	+5	4
3 Bosnia y Herzegovina	3	-1	4
4 Catar	3	-8	1

Grupo C

Equipo	PJ	DG	PTS
1 Brasil	3	+6	7
2 Marruecos	3	+3	7
3 Escocia	3	-3	3
4 Haití	3	-6	0

Grupo D

Equipo	PJ	DG	PTS
1 Estados Unidos	3	+4	6
2 Australia	3	0	4
3 Paraguay	3	-2	4
4 Turquía	3	-2	3

Grupo E

Equipo	PJ	DG	PTS
1 Alemania	3	+6	6
2 Costa de Marfil	3	+2	6
3 Ecuador	3	0	4
4 Curacao	3	-8	1

Grupo F

Equipo	PJ	DG	PTS
1 Países Bajos	3	+6	7
2 Japón	3	+4	5
3 Suecia	3	0	4
4 Túnez	3	-10	0

Grupo G

Equipo	PJ	DG	PTS
1 Bélgica	3	+4	5

Grupo H

Equipo	PJ	DG	PTS
1 España	3	+5	7

Grupo I

Equipo	PJ	DG	PTS
1 Francia	3	+8	9

Vista de clasificaciones: panel de grupos con puntos, diferencia de goles y partidos jugados.

Criterio	Decisión aplicada
Legibilidad	Titulos grandes, contraste alto, tablas con pocas columnas y prioridad visual para puntos y posicion.
Consistencia	Mismo lenguaje visual en calendario, selecciones y clasificaciones.
Escalabilidad	La interfaz soporta doce grupos y cuarenta y ocho selecciones mediante filtros y tarjetas.
Feedback visual	Estados activos en navegación, etiquetas de grupo, botones principales y separadores ligeros.
Portfolio	La presentación final comunica tanto capacidad técnica como sensibilidad de producto y acabado visual.

Criterio de validación

- La interfaz final permite seguir el calendario, consultar plantillas y controlar la clasificación de los doce grupos.
- Las vistas mantienen coherencia visual entre contenido dinámico, filtros y formularios.
- La mejora de CSS no altera la arquitectura de datos ni la lógica principal de la API.

5. Proceso de trabajo y uso de IA

El desarrollo se planteo por capas. Primero se definio el modelo de datos, después se implemento la API y finalmente se conecto el cliente web mediante peticiones asíncronas. En la fase final se trabajo el diseño visual del cliente con apoyo de Codex e IA, principalmente para proponer, comparar y ajustar reglas CSS.

Fase	Descripción
Modelo de datos	Creación de tablas para entrenadores, estadios, selecciones, jugadores y partidos, con relaciones entre entidades.
API REST	Implementación de rutas limpias en PHP, validaciones, consultas preparadas con PDO y respuestas JSON normalizadas.
Cliente web	Construccion de interfaz con tarjetas, tablas, filtros y formularios usando HTML, CSS y JavaScript vanilla.
Integración	Uso de XMLHttpRequest para consultar endpoints, enviar formularios y actualizar zonas de contenido.
Refinamiento CSS con IA	Generacion de alternativas de layout, jerarquía visual, espaciado, sombras, tarjetas, sidebar y estilos de tablas; posterior selección y adaptación manual.
Preparacion para portfolio	Limpieza de credenciales, creación de config.example.php, mejora del README y subida del repositorio a GitHub.

Uso responsable de Codex e IA

- La IA se utilizó como herramienta de apoyo para acelerar iteraciones de diseño CSS y explorar soluciones visuales.
- Las decisiones finales de estructura, integración, pruebas y adecuación al proyecto se revisaron manualmente.
- No se delegó la lógica de negocio sin validación: el comportamiento de API, base de datos y cliente se comprobó sobre el proyecto real.
- El resultado documenta tanto el desarrollo técnico como el proceso de mejora visual asistido.

Estructura del repositorio

```
mundial/  
|-- api/  
|   |-- inc/  
|   |   |-- config.example.php  
|   |   |-- database.php  
|   |-- .htaccess  
|   |-- index.php  
|-- css/style.css  
|-- js/script.js  
|-- sql/  
|   |-- mundial.sql  
|   |-- partidos.sql  
|-- index.html  
|-- .gitignore  
|-- README.md
```

6. Resultados finales

El resultado es una aplicación web académica funcional, con separación clara entre interfaz, lógica de API y persistencia. Como pieza de portfolio, evidencia conocimientos de desarrollo web, comunicación cliente-servidor, diseño de bases de datos, control de versiones con Git/GitHub y capacidad para mejorar la experiencia visual del producto.

Resultado	Detalle
Interfaz web	Página HTML con estilos CSS renovados, tarjetas de selecciones, tablas de clasificación, listados, filtros, sidebar y formularios.
Backend	API REST en PHP con validaciones, consultas preparadas, control de errores y respuestas JSON.
Base de datos	Scripts SQL para crear e inicializar la información del torneo y partidos de ejemplo.
Repositorio publico	Código preparado para GitHub con README, .gitignore y archivo de configuración de ejemplo.
Documentación	PDF final que incorpora arquitectura, funcionalidades, capturas finales y declaración del uso de IA en el proceso.

Entregables

- Repositorio GitHub: <https://github.com/alexpunzi/mundial-2026-api-web>
- Cliente web: index.html, css/style.css y js/script.js.
- API REST: api/index.php, api/inc/database.php y configuración local mediante config.php.
- Base de datos: sql/mundial.sql y sql/partidos.sql.
- Documentación: README.md y este PDF final de presentación del proyecto.

Posibles mejoras futuras

- Sustituir XMLHttpRequest por fetch con async/await para modernizar el cliente.
- Separar la API en controladores, servicios y repositorios para mejorar mantenibilidad.
- Añadir autenticación para proteger operaciones de escritura.
- Incorporar tests de endpoints y validaciones automaticas.
- Preparar un despliegue real en hosting con PHP y MySQL o migrar el backend a una plataforma desplegable.
- Documentar con más detalle las decisiones de diseño y crear una pequeña guia de componentes CSS.

7. Conclusion

Este proyecto consolida un flujo básico de desarrollo web completo: diseño de estructura de datos, implementación de API, consumo desde el cliente, publicación en GitHub y mejora visual de la interfaz. Frente a un proyecto puramente visual, aporta evidencia técnica de programación de cliente, backend PHP, SQL relacional y buenas prácticas iniciales de preparación de repositorios para portfolio. La incorporación de IA queda integrada como apoyo de productividad y refinamiento de CSS, con supervisión humana y validación sobre el producto final.